ment-arrays, or just a list of arguments, in which case Ruby assumes the keys and values alternate.

The method Hash.new will create an empty hash. This method takes an optional argument that acts as the default value for keys that are not in the hash. In this usage, the default value is not duplicated each time it's used, so you should use an immutable value, like a scalar, and not a mutable value, like an empty array. So, this will not do what you want:

```
sample = Hash.new([])
sample[:a] << "alpha" # => ["alpha"]
sample[:b] << "beta"  # => ["alpha", "beta"]
sample[:a]            # => ["alpha", "beta"]
sample[:b]            # => ["alpha", "beta"]
```

The idea here is to bypass having to create an array for each new key and just use a default, but in this version, only one array is used for the default so adding values to it in different keys, just updates the single default value.

Alternately, you can pass a block, which is invoked when a key not in the hash is requested. The block takes two arguments—the hash and the new key—and returns a default value for the key. This enables you to create a new default object for each key:

```
sample = Hash.new { |hash, key| hash[key] = [] }
sample[:a] << "alpha" # => ["alpha"]
sample[:b] << "beta"  # => ["beta"]
sample[:a]            # => ["alpha"]
sample[:b]            # => ["beta"]
```

Hash Values

Hash values are accessed using the [] method:

```
hash = {a: 1, "b": 2, c: 3}
hash[:a]  # => 1
```

The value associated with the key is returned. Ruby uses eql? (or ==) to determine the key. Symbol keys are not matched by string values (although you may see this in Ruby on Rails, which has a popular extension to allow string and symbol keys to overlap).

If there is no matching key and the hash has a default value, Ruby will return the default. If there is no matching key and the hash has a default block, Ruby will invoke the block and return the resulting value. If the hash has no defaults, Ruby will return nil.

Ruby does have a way to retrieve values with different semantics. The fetch method takes a key and an optional default. The default is either a second positional argument or a block that takes a single argument, the key. If fetch is called with a key that is not in the hash, it returns the default. If there is no default, fetch raises a KeyError.

So we could write our earlier example using fetch like this:

```
sample = {}
sample[:a] = sample.fetch(:a, []) << "alpha"
sample[:b] = sample.fetch(:b, []) << "beta"
sample[:a] # => ["alpha"]
sample[:b] # => ["beta"]
```